

Phase 1: The Core Foundation (Fundamentals)

Before jumping into advanced optimization, these are the absolute essentials you need to build any basic app.

- **Component Design:** How to split your UI into small, reusable building blocks.
- **Props:** Passing data from a parent component down to a child component.
- **State & useState:** Managing data that changes inside a component over time.
- **Folder Structure:** Organizing your files (e.g., grouping by feature or component types) so your project stays clean as it grows.

Phase 2: Side Effects & Advanced State

Once you can build static layouts and handle basic clicks, you need to handle real-world data and complex logic.

- **Lifecycle Hooks & useEffect:** Running code at specific times, like fetching data from an API when the page loads or cleaning up timers.
- **useRef:** Grabbing a direct hold of a DOM element or storing values that don't trigger a re-render when they change.
- **useContext:** Sharing global data (like a dark mode theme or user login info) across many components without passing props down manually through every level.
- **useReducer:** Managing complex state logic where multiple things can happen to the same data (an alternative to multiple `useState` calls).

Phase 3: Routing & App Architecture

This phase turns a single-page concept into a fully functional multi-page web application.

- **Routing:** Navigating between different pages (like `/home` to `/dashboard`) without reloading the entire browser.
- **Protected Routes:** Blocking unauthorized users from seeing specific pages (e.g., redirecting to `/login` if not authenticated).
- **Forms & Validation:** Handling user inputs safely and showing error messages if they type something wrong.

Phase 4: Performance & Advanced Patterns

This is where you transition from a developer who just makes things work to an engineer who makes things fast and resilient.

- **useMemo & useCallback:** Preventing unnecessary recalculations and re-renders to keep your app running smoothly.
- **Lazy Loading & Suspense:** Splitting your code into smaller pieces so users only download the code for the page they are currently looking at.
- **Error Boundaries:** Catching JavaScript errors in components so the whole app doesn't crash into a blank white screen.
- **Reusable Components & Custom Hooks:** Extracting repetitive logic into your own custom functions so you can use them anywhere.